Computer Architecture Lab 1

Jonas Van Nieuwenhuyze, Gil Nimmegeers

1 Introduction

This lab explains how to flip an array using the NVIDIA GPU. Unlike conventional CPU architectures optimized for sequential processing, GPUs are designed to efficiently handle massive parallel workloads. Through platforms like CUDA, developers can use the thousands of parallel processing cores in GPUs to tackle computationally intensive tasks efficiently.

2 Implementation

The code is divided into two parts: the host code (executed on the CPU) and the kernel function (executed on the GPU). The host code initializes an array of integers on the CPU, allocates memory on the GPU for two arrays (one for the input and one for the output), and copies the input array from the CPU to the GPU. In GPU programming, it's a good practice to use separate arrays for input and output to prevent synchronization errors. Synchronization errors occur when multiple threads attempt to write to the same memory location simultaneously, leading to data corruption or undefined behavior. By utilizing separate input and output arrays, each thread in the GPU kernel can independently write to its designated output location without conflicting with other threads.
The kernel function is then launched on the GPU. This function is responsible for performing the actual computation task. Each element of the output array is calculated independently by a separate thread running on the GPU. The block size is set to the maximum of 1024 threads. Every element in the array should correspond with one thread. So if the array size is larger than 1024, more than one block is needed. If the array size is not a multiple of the block size, the last block is not fully used. Therefore, there is a check in the kernel function to make sure the GPU is not writing to or reading from uninitialised memory.
After the kernel function has finished executing on the GPU, the output array is copied back from the GPU to the CPU. Finally, the flipped array is printed out and all the memory is released.

3 Conclusion

Overall, the program demonstrates how to harness the parallel processing power of a GPU to accelerate computation-intensive tasks, such as array manipulation, by executing them concurrently across multiple processing units within the GPU.